PROJECT ARC — MULTI-USER FIREBASE MIGRATION

READ-ONLY VERIFICATION & AUDIT REPORT (PAGE 1 OF 5)

CRITICAL SAFETY MANDATE — READ-ONLY AUDIT GUARANTEE
This audit is strictly read-only. No Flutter code was modified. No Firebase Authentication accounts were created/deleted. No Firestore documents or Security Rules were altered. Zero write operations were executed against live Firebase or local disk.
AUDIT COMPLETE — NO PROJECT OR FIREBASE DATA WAS MODIFIED.

1. Part 2 — Firebase Project Identification & Config Verification

Authoritative analysis cross-referenced configuration manifests (firebase_options.dart, .firebaserc, google-services.json) with live GCP resources via API inspection.

Parameter	Value Discovered in Manifests & Live GCP	Verification Status
Firebase Project Name	Project Arc	■ Verified Live
Firebase Project ID	**arc-productivity-shreyas**	■ Verified Live
Firebase Project Number	729643761637	■ Verified Live
Android App ID	1:729643761637:android:39bc738c98a05bd4dc790c	■ Verified Live
iOS App ID	1:729643761637:ios:1f1fa53c132914cbdc790c	■ Verified Live
Android Package ID	com.arc.arc	■ Verified Code

Documentation Discrepancy Note: Earlier reports (Step 4 & 5) referenced Project ID shreyas-84524. Live GCP inspection confirms the real project ID is **arc-productivity-shreyas**. shreyas-84524 is the repository workspace folder name.

2. Part 3 & 14 — Firebase Authentication Architecture (Option A)

The application implements Option A: User inputs friendly username ('Shreyas') & password → App maps to shreyas@arc.app → Authenticates with Firebase Auth → Generates 28-char UID → Injects into Riverpod UserContext.

Username	Internal Email Identity	Intended Password	Live Firebase Auth Status
Shreyas	shreyas@arc.app	123456 (Managed by Auth)	■ Not Created (0 in Auth)
Aman	aman@arc.app	123456 (Managed by Auth)	■ Not Created (0 in Auth)
Rohit	rohit@arc.app	123456 (Managed by Auth)	■ Not Created (0 in Auth)
Sachi	sachi@arc.app	123456 (Managed by Auth)	■ Not Created (0 in Auth)
Yash	yash@arc.app	123456 (Managed by Auth)	■ Not Created (0 in Auth)

3. Part 4 — Live Firebase Auth Audit & Firebase Console Explanation

Direct API queries (auth_get_users) returned **0 accounts** on the live backend. Previous migration steps executed using offline/mock fallback UIDs (e.g. arc_uid_shreyas), so zero users were registered on cloud servers.
Firebase Console View: Navigate to *Firebase Console → Project Arc → Build → Authentication → Users*. You will see an **empty table** ('No users found'). When real users log in, the table will display shreyas@arc.app in the Identifier column (not friendly username 'Shreyas').

PROJECT ARC — MULTI-USER FIREBASE MIGRATION

READ-ONLY VERIFICATION & AUDIT REPORT (PAGE 2 OF 5)

4. Part 5 — Live Cloud Firestore Database Hierarchy Audit

Direct API inspection (firestore_list_collections & firestore_list_documents) was executed on database projects/arc-productivity-shreyas/databases/(default).

Database Element	Codebase Design Expectation	Live Backend Reality Discovered	Audit Finding
Collection users	Root path for user subcollections	**DOES NOT EXIST**	■ Unpopulated on Cloud
Subcollection tasks	users/{uid}/tasks	Root collection /tasks (17 items)	■ Stored in Legacy Root
Subcollection habits	users/{uid}/habits	Root collection /habits (12 items)	■ Stored in Legacy Root
Subcollection habitEntries	users/{uid}/habitEntries	Root collection /habitEntries (60 items)	■ Stored in Legacy Root
Subcollection transactions	users/{uid}/transactions	Root collection /transactions (8 items)	■ Stored in Legacy Root
Subcollection budgets	users/{uid}/budgets	Root collection /budgets (2 items)	■ Stored in Legacy Root
Subcollection monthlySummaries	users/{uid}/monthlySummaries	Root collection /monthlySummaries (2 items)	■ Stored in Legacy Root
Subcollection settings	users/{uid}/settings/profile	Root doc /settings/global (1 item)	■ Stored in Legacy Root

5. Part 6 — User to UID & Firestore Path Mapping Matrix

The complete mapping between friendly usernames, internal credentials, code UIDs, and live Firestore paths:

User	Internal Email	Code Fallback UID	Live Auth UID	Target Firestore Path	Live Status
Shreyas	shreyas@arc.app	arc_uid_shreyas	Pending Login	users/{shreyasUid}/*	■ Not in DB
Aman	aman@arc.app	arc_uid_aman	Pending Login	users/{amanUid}/*	■ Not in DB
Rohit	rohit@arc.app	arc_uid_rohit	Pending Login	users/{rohitUid}/*	■ Not in DB
Sachi	sachi@arc.app	arc_uid_sachi	Pending Login	users/{sachiUid}/*	■ Not in DB
Yash	yash@arc.app	arc_uid_yash	Pending Login	users/{yashUid}/*	■ Not in DB

6. Data Category Schemas & Location Definitions

- **Tasks**: ID (UUID), title, description, priority, category, dueDate, dueTime, completed, createdAt, updatedAt.
- **Habits**: ID (UUID), name, category, colorValue, reminderTime, goalType, createdAt, updatedAt.
- **Habit Entries**: ID (${habitId}_${dateStr}), habitId, dateStr, completed.
- **Transactions**: ID (UUID), amount, type (Expense/Income), category, notes, date, time, createdAt.
- **Budgets**: Month (YYYY-MM), amount.
- **Monthly Summaries**: Month (YYYY-MM), totalIncome, totalExpenses, savings, monthlyBudget.

PROJECT ARC — MULTI-USER FIREBASE MIGRATION

READ-ONLY VERIFICATION & AUDIT REPORT (PAGE 3 OF 5)

7. Part 7 — Security Rules Audit: Code vs Deployed Live Firebase

A direct comparison was conducted between the local repository rules file (firestore.rules) and the security rules actively running on the live Cloud Firestore database (firebase_get_security_rules).

Location	Active Security Rules Source Code	Security & Isolation Assessment

Local Code Repo (firestore.rules)	`rules_version = '2';`
	`service cloud.firestore {`
	`  match /databases/{database}/documents {`
	`  match /users/{userId}/{document=**} {`
	`  allow read, write: if request.auth != null && request.auth.uid == userId;`
	`  }`
	`  match /tasks/{docId} { allow read: if request.auth != null; }`
	`  ... legacy collections authenticated read ...`
	`  }`
	`}`
		■ **Hardened Multi-Tenant Isolation** Strict UID scoping. User A cannot read/write User B's subcollections.

Live Backend (Deployed Rules)	`rules_version = '2';`
	`service cloud.firestore {`
	`  match /databases/{database}/documents {`
	`  match /{document=**} {`
	`  allow read, write: if true;`
	`  }`
	`  }`
	`}`
		■ **UNPROTECTED TEST MODE** Public read/write access. Hardened rules were **NEVER DEPLOYED** via Firebase CLI.

8. User Data Isolation Enforcement Flow

When the hardened rules in firestore.rules are deployed, user isolation will be enforced dynamically at the database level by the Cloud Firestore engine:

AUTHENTICATION & AUTHORIZATION CONTROL PIPELINE
*1. **Client Request**: App issues read/write to users/abc123xyz/tasks/task_01 with Auth JWT Token.*
*2. **Auth Check**: Firestore asserts request.auth != null. (Fails if unauthenticated → 403 Error).*
*3. **UID Match**: Firestore compares token UID (request.auth.uid) with path variable (userId == 'abc123xyz').*
*4. **Authorization Decision**: If UID matches, access GRANTED. If User B attempts to query User A's path, access REJECTED.*

9. Part 8 — Legacy Collections Protection Status

• **Current Live Status**: Legacy root collections (/tasks, /habits, etc.) are currently publicly readable/writable because the live rules are set to allow read, write: if true;.
• **Target Status**: Once firestore.rules is deployed, root collections will be restricted to authenticated read-only access (allow read: if request.auth != null;). This prevents unauthenticated public access while keeping legacy data accessible as a read-only safety backup.

PROJECT ARC — MULTI-USER FIREBASE MIGRATION

READ-ONLY VERIFICATION & AUDIT REPORT (PAGE 4 OF 5)

10. Part 9 — Shreyas Historical Single-User Data Inventory

An empirical document-by-document audit was executed across all root collections in live Cloud Firestore to verify Shreyas's original data created prior to multi-user migration.

Collection Name	Live Count (Root Path)	Sample Document Details / Content Verified	Count in users/{uid}/*
/tasks	**17 docs**	Registration Posters, SiH details Nikalo, ARC app changes, Event Logo, Story Templates, Sponsor Brochure	0 docs
/habits	**12 docs**	College, Vit. D, FW2, GYM, Creatine, FW1, Hydration, Vit. E, No Junk Food, No P, No M, FW3	0 docs
/habitEntries	**60 docs**	Daily completion check-in logs from July & August 2026 for all 12 habits	0 docs
/transactions	**8 docs**	Misal Paav (■65), Football Turf (■200), Brand New Day (■335), Masala Peanuts (■10), Shawrma (■140)	0 docs
/budgets	**2 docs**	2026-07 (■200 budget), 2026-08 (■1,000 budget)	0 docs
/monthlySummaries	**2 docs**	2026-07 (Expenses ■65), 2026-08 (Expenses ■880)	0 docs
/settings	**1 doc**	settings/global (Theme mode, notification preferences)	0 docs

Why Shreyas Data is Still in Root: The migration service (DataMigrationService) is programmed to run non-destructively upon Shreyas's first authenticated production login. Because no real Firebase Auth login has occurred on the live app, **100% of Shreyas's data remains perfectly preserved in root collections**. Zero data was lost.

11. Part 10 — Hive Local Storage vs Cloud Firestore Architecture

It is essential to distinguish local device storage (Hive) from cloud database storage (Firestore):

Architectural Dimension	Local Device Storage (Hive)	Cloud Backend Database (Firestore)
Storage Location	Local binary files on Android/iOS disk	Google Cloud Firestore NoSQL Server
Primary Purpose	Instant app loading, offline cache, offline queue	Cloud backup, multi-device sync, data recovery
Multi-User Isolation	User-scoped boxes: tasks_box_{uid}, habits_box_{uid}, money_box_{uid}	User-scoped subcollections: users/{uid}/tasks, users/{uid}/habits
Legacy Fallback	Legacy default boxes: tasks_box, habits_box	Legacy root collections: /tasks, /habits
Visibility	Invisible on Firebase Console (Local client file)	Visible on Firebase Console website

PROJECT ARC — MULTI-USER FIREBASE MIGRATION

READ-ONLY VERIFICATION & AUDIT REPORT (PAGE 5 OF 5)

12. Part 13 — Step-by-Step Firebase Console Independent Verification Guide

1. Open **console.firebase.google.com** and select project **Project Arc** (ID: arc-productivity-shreyas).
2. Go to **Build** → **Authentication** → **Users** tab → Observe empty list ('No users found').
3. Go to **Build** → **Firestore Database** → **Data** tab → Observe 7 root collections: budgets, habitEntries, habits, monthlySummaries, settings, tasks, transactions.
4. Confirm root collection users DOES NOT EXIST.
5. Click /tasks → Inspect docs (e.g. 064b92... 'Registration Posters') → Verify 17 tasks created by Shreyas.
6. Click /habits → Inspect docs → Verify 12 habits ('GYM', 'Vit. D', 'Hydration', etc.).
7. Click /transactions → Inspect docs → Verify 8 expenses ('Misal Paav' ■65, 'Football Turf' ■200).
8. Go to **Build** → **Firestore Database** → **Rules** tab → Verify active rules are allow read, write: if true;.

13. Part 11 & 12 — Migration Verification & Code vs Backend Comparison

Feature / Step	What Flutter Code Claims	What Live Firebase Contains	Final Verification Status
Step 1: Architecture Foundation	UserContext & Profile models built	Project arc-productivity-shreyas connected	■ Code Ready / Rules Unpushed
Step 2: Data Foundation	User-scoped repos & Hive isolation	Cloud Firestore DB unpopulated under users/	■ Code Ready / DB Unpopulated
Step 3: Migration Dry-Run	Dry-run verification logic built	Cloud migration to users/{uid}/* never run	■ Not Executed on Backend
Step 4: Firebase Authentication	Option A username mapping active	0 users registered in Firebase Auth	■ 0 Accounts in Live Auth
Step 5: Security Hardening	Change password & hardened rules	Active rules are allow read, write: if true;	■ Hardened Rules Not Deployed
Shreyas Data Migration	Copied to users/{shreyasUid}/*	100% of data remains in root collections	■ Safe in Root / Pending Login

14. Part 15 — Final Audit Verdict & Actionable Deployment Recommendations

SYSTEM VERDICT: ■ NOT VERIFIED ON LIVE FIREBASE BACKEND (■ PARTIALLY VERIFIED IN CODEBASE)
Rationale: The Flutter codebase is 100% feature-complete with Option A auth, user-scoped repositories, Hive isolation, and migration services. However, the multi-user system has **never been applied to the live Firebase project**. 0 accounts exist in Auth, 0 docs exist in users/, and live security rules remain in test mode.
Actionable Deployment Plan:
1. Run firebase deploy --only firestore:rules to push hardened Security Rules to production.
2. Launch app and log in as Shreyas (Password 123456) → Automatically provisions shreyas@arc.app in Auth and migrates all 17 tasks, 12 habits, 60 entries, and 8 transactions to users/{shreyasUid}/*.
3. Log in as Aman, Rohit, Sachi, and Yash to provision clean user namespaces.

AUDIT COMPLETE — NO PROJECT OR FIREBASE DATA WAS MODIFIED.